เอกสารวิธีทำแบบฝึกหัด: เพิ่ม Review ให้ Book

Prisma + SQLite ฉบับ Express และ Next.js (App Router) พร้อมคำอธิบายทุกขั้นตอน

1. ภาพรวมโจทย์

โจทย์ให้เพิ่มตาราง `Review` ที่เชื่อมกับ `Book` แบบ **1 ต่อ หลาย (1-to-Many)** คือหนังสือ 1 เล่มมีได้หลายรีวิว แต่รีวิว 1 อันเป็นของหนังสือเพียงเล่มเดียว

โครงสร้างข้อมูล (ER Diagram)

Book	ความสัมพันธ์	Review
id (PK)	1 ต่อ หลาย	id (PK)
title (จำเป็น)		rating (1-5)
author (จำเป็น)		content
publishedAt (ค่าเริ่มต้น now)		bookId (FK ชี้ไป Book.id)
coverUrl (ว่างได้)		

งานที่ต้องทำ 4 ขั้นตอน

ขั้น	งาน	สิ่งที่ต้องทำ
1	Schema Extension	แก้ `schema.prisma` เพิ่ม model Review และความสัมพันธ์กับ Book
2	Database Migration	รัน `npx prisma db push` เพื่อสร้างตารางใน SQLite
3	API Implementation	เพิ่ม endpoint สำหรับโพสต์รีวิวของหนังสือแต่ละเล่ม
4	Data Retrieval	แก้การดึงหนังสือให้ได้รีวิวซ้อนมาด้วย

เอกสารนี้มี 2 ฉบับให้เลือกทำตาม คือ **ส่วน Express** (หัวข้อ 5) และ **ส่วน Next.js** (หัวข้อ 6) ขั้นที่ 1-2 เหมือนกันทั้งสองแบบ

2. สิ่งที่ต้องเตรียม

- Node.js เวอร์ชัน 18 ขึ้นไป (เช็คด้วย `node -v`)
- npm และโปรแกรมแก้ไขโค้ด เช่น VS Code
- โปรแกรมสำหรับทดสอบ API เช่น `curl`, Postman หรือ Thunder Client

> **หมายเหตุเรื่องเวอร์ชัน Prisma:** เอกสารนี้ใช้ Prisma 6 (`prisma-client-js`) เพราะ schema และโค้ดตรงกับที่โจทย์ใช้ ถ้าติดตั้ง Prisma 7 ขึ้นไป รูปแบบ generator และการเชื่อมต่อฐานข้อมูลจะต่างออกไป จึงให้ระบุเวอร์ชัน `@6` ตอนติดตั้งตามคำสั่งด้านล่าง

3. เริ่มโปรเจกต์ (ฉบับ Express)

ถ้าทำฉบับ Next.js ให้ข้ามไปดูหัวข้อ 6.1 แล้วกลับมาทำหัวข้อ 4 ต่อ

```
mkdir book-express
cd book-express
npm init -y
npm install express @prisma/client@6
npm install -D prisma@6
npx prisma init --datasource-provider sqlite
```

อธิบายคำสั่ง

คำสั่ง	ความหมาย
`npm init -y`	สร้าง `package.json` ด้วยค่าเริ่มต้น
`npm install express @prisma/client@6`	ติดตั้ง Express และ Prisma Client (ตัวที่โค้ดเรียกใช้ฐานข้อมูล)
`npm install -D prisma@6`	ติดตั้ง Prisma CLI (เครื่องมือสั่งงาน) เป็น devDependency
`npx prisma init --datasource-provider sqlite`	สร้างโฟลเดอร์ `prisma/` กับไฟล์ `schema.prisma` และไฟล์ `.env` โดยตั้งให้ใช้ SQLite

หลังรันเสร็จจะได้ไฟล์ `.env` ที่มีบรรทัด `DATABASE_URL="file:./dev.db"` ซึ่งหมายถึงให้สร้างไฟล์ฐานข้อมูล `dev.db` ไว้ในโฟลเดอร์ `prisma/`

ไฟล์ `prisma/schema.prisma` ที่ได้จะเป็นโครงเปล่า ให้เพิ่ม model `Book` ตามภาพโจทย์ก่อน (ถ้าโปรเจกต์ของคุณมี Book อยู่แล้วให้ข้ามไปขั้นที่ 1 ได้เลย)

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

model Book {
  id          Int      @id @default(autoincrement())
  title       String
  author      String
  publishedAt DateTime @default(now())
  coverUrl    String?
}
```

4. ขั้นที่ 1-2: แก้ Schema และ Migration (ใช้ร่วมกันทั้ง Express และ Next.js)

ขั้นที่ 1: Schema Extension

เปิด `prisma/schema.prisma` แล้วแก้ model `Book` และเพิ่ม model `Review` ดังนี้

```
model Book {
  id          Int      @id @default(autoincrement())
  title       String
  author      String
  publishedAt DateTime @default(now())
  coverUrl    String?
  reviews     Review[]
}

model Review {
  id      Int    @id @default(autoincrement())
  rating  Int
  content String
  bookId  Int
  book    Book   @relation(fields: [bookId], references: [id], onDelete:
Cascade)
}
```

อธิบายทีละบรรทัด

โค้ด	ความหมาย
`@id`	กำหนดเป็น Primary Key (กุญแจหลักที่ไม่ซ้ำกัน)
`@default(autoincrement())`	ให้ฐานข้อมูลรันเลข 1, 2, 3 ... ให้อัตโนมัติ ไม่ต้องส่ง id เอง
`String?`	เครื่องหมาย `?` แปลว่า **Nullable** คือเว้นว่างได้ (ตรงกับ `coverUrl` ในภาพ)
`@default(now())`	ใส่วันเวลาปัจจุบันให้อัตโนมัติตอนสร้างข้อมูล
`reviews Review[]`	ฝั่ง "หลาย" ของ Book บอกว่าหนังสือมีรีวิวได้หลายอัน **ไม่ได้สร้างคอลัมน์จริง** ในตาราง เป็นแค่ช่องทางให้ Prisma ดึงรีวิวมาให้
`bookId Int`	**Foreign Key** คอลัมน์จริงที่เก็บ id ของหนังสือเจ้าของรีวิว
`book Book @relation(fields: [bookId], references: [id])`	บอก Prisma ว่าคอลัมน์ `bookId` ชี้ไปที่ `Book.id`
`onDelete: Cascade`	ถ้าลบหนังสือ รีวิวทั้งหมดของเล่มนั้นจะถูกลบตามไปด้วย ป้องกันรีวิวที่ไม่มีเจ้าของค้างในระบบ

ทำไมต้องเขียน relation ทั้ง 2 ฝั่ง

Prisma บังคับให้ประกาศความสัมพันธ์ครบสองด้าน คือ `reviews Review[]` ใน Book และ `book Book @relation(...)` ใน Review ถ้าขาดด้านใดด้านหนึ่งจะขึ้น error ตอนตรวจ schema

เรื่อง rating 1-5

Prisma ไม่มีคำสั่งจำกัดช่วงตัวเลขใน schema ดังนั้นต้องตรวจค่า `rating` ในโค้ด API เอง (ดูขั้นที่ 3)

ตรวจความถูกต้องของ schema

```
npx prisma validate
npx prisma format
```

`validate` ตรวจว่า schema ถูกต้อง ส่วน `format` จัดย่อหน้าให้สวยงาม

ขั้นที่ 2: Database Migration

```
npx prisma db push
```

คำสั่งนี้ทำอะไร

1. อ่าน `schema.prisma` แล้วเปรียบเทียบกับฐานข้อมูล SQLite
2. สร้างตาราง `Review` พร้อม Foreign Key ที่ชี้ไป `Book`
3. สร้างไฟล์ฐานข้อมูล `prisma/dev.db` ถ้ายังไม่มี
4. สร้าง Prisma Client ใหม่ให้อัตโนมัติ ถ้าไม่ทำงาน ให้รัน `npx prisma generate` เพิ่ม

`db push` ต่างจาก `migrate dev` อย่างไร

	`prisma db push`	`prisma migrate dev`
เก็บประวัติ migration	ไม่เก็บ	เก็บเป็นไฟล์ SQL ในโฟลเดอร์ `migrations/`
เหมาะกับ	เรียนรู้ ทดลอง ต้นแบบ	โปรเจกต์จริงที่ทำงานเป็นทีม
ความเร็ว	เร็ว ไม่ต้องตั้งชื่อ	ต้องตั้งชื่อ migration

โจทย์สั่งให้ใช้ `db push` จึงเหมาะกับการเรียนรู้ที่ไม่ต้องการประวัติการแก้ฐานข้อมูล

ตรวจผลลัพธ์ด้วย Prisma Studio

```
npx prisma studio
```

เบราว์เซอร์จะเปิดหน้าจัดการข้อมูล ซึ่งควรเห็นตาราง `Book` และ `Review` เพิ่มข้อมูลทดลองได้จากหน้านี้เลย

เตรียมข้อมูลตัวอย่าง (Seed)

เพื่อให้ทดสอบ API ได้ ควรมีหนังสืออย่างน้อย 1 เล่มในฐานข้อมูล สร้างไฟล์ `prisma/seed.js`

```
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();

async function main() {
  await prisma.book.create({
    data: {
      title: 'Clean Code',
      author: 'Robert C. Martin',
      reviews: {
```

```
        create: [
          { rating: 5, content: 'ดีมาก อ่านแล้วเขียนโค้ดดีขึ้น' },
          { rating: 4, content: 'เนื้อหาแน่น แต่บางบทยาว' },
        ],
      },
    },
  });
  await prisma.book.create({
    data: { title: 'The Pragmatic Programmer', author: 'Andrew Hunt' },
  });
  console.log('seed เสร็จแล้ว');
}

main()
  .catch((e) => console.error(e))
  .finally(() => prisma.$disconnect());
```

รันด้วย

```
node prisma/seed.js
```

จุดที่ควรสังเกตคือ `reviews: { create: [...] }` เป็นการสร้างหนังสือพร้อมรีวิวลูกในคำสั่งเดียว (nested write) Prisma จะใส่ `bookId` ให้เองโดยอัตโนมัติ

5. ฉบับ Express: ขั้นที่ 3-4

โครงสร้างไฟล์

```
book-express/
├─ prisma/
│  ├─ schema.prisma
│  ├─ seed.js
│  └─ dev.db
├─ .env
├─ index.js
└─ package.json
```

สร้างไฟล์ `index.js`

```
const express = require('express');
const { PrismaClient } = require('@prisma/client');

const prisma = new PrismaClient();
const app = express();

app.use(express.json());

// ---------- ขั้นที่ 3: เพิ่มรีวิวให้หนังสือ ----------
app.post('/books/:id/reviews', async (req, res) => {
  try {
    const bookId = Number(req.params.id);
    const { rating, content } = req.body;

    if (!Number.isInteger(bookId)) {
      return res.status(400).json({ error: 'id ไม่ถูกต้อง' });
    }
    if (!Number.isInteger(rating) || rating < 1 || rating > 5) {
      return res.status(400).json({ error: 'rating ต้องเป็นจำนวนเต็ม 1-5' });
    }
    if (typeof content !== 'string' || content.trim() === '') {
      return res.status(400).json({ error: 'ต้องมี content' });
    }

    const book = await prisma.book.findUnique({ where: { id: bookId } });
    if (!book) {
      return res.status(404).json({ error: 'ไม่พบหนังสือ' });
    }

    const review = await prisma.review.create({
      data: { rating, content: content.trim(), bookId },
    });

    res.status(201).json(review);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'เกิดข้อผิดพลาดที่เซิร์ฟเวอร์' });
  }
});

// ---------- ขั้นที่ 4: ดึงหนังสือพร้อมรีวิว ----------
```

```
app.get('/books', async (req, res) => {
  const books = await prisma.book.findMany({
    include: { reviews: true },
    orderBy: { publishedAt: 'desc' },
  });
  res.json(books);
});

app.get('/books/:id', async (req, res) => {
  const book = await prisma.book.findUnique({
    where: { id: Number(req.params.id) },
    include: { reviews: { orderBy: { id: 'desc' } } },
  });
  if (!book) return res.status(404).json({ error: 'ไม่พบหนังสือ' });
  res.json(book);
});

const PORT = 3000;
app.listen(PORT, () => console.log(`เซิร์ฟเวอร์ทำงานที่ http://localhost:${PORT}`));
```

อธิบายขั้นที่ 3 (POST รีวิว)

ส่วนของโค้ด	เหตุผล
`app.use(express.json())`	ทำให้ Express อ่าน body แบบ JSON ได้ ถ้าลืมบรรทัดนี้ `req.body` จะเป็น `undefined`
`Number(req.params.id)`	ค่าใน URL เป็น string เสมอ ต้องแปลงเป็นตัวเลขก่อนส่งให้ Prisma
ตรวจ `rating` ด้วย `Number.isInteger`	กันค่าที่ไม่ใช่จำนวนเต็ม เช่น 3.5 หรือ "5" และกันค่านอกช่วง 1-5 เพราะฐานข้อมูลไม่ได้ตรวจให้
ตรวจ `content`	กันค่าว่างและค่าที่ไม่ใช่ข้อความ
`findUnique` เช็คหนังสือก่อน	ถ้าไม่เช็ค Prisma จะโยน error เรื่อง Foreign Key ที่อ่านยาก การตอบ 404 ชัดเจนกว่า
`prisma.review.create`	สร้างแถวใหม่ในตาราง Review
`status(201)`	201 Created คือรหัสมาตรฐานเมื่อสร้างข้อมูลสำเร็จ
`try / catch`	จับ error ที่คาดไม่ถึง และตอบ 500 แทนที่เซิร์ฟเวอร์จะล่ม

วิธีเขียนอีกแบบ: ใช้ `connect`

```
await prisma.review.create({
  data: { rating, content, book: { connect: { id: bookId } } },
});
```

`connect` คือการบอก Prisma ว่าให้ผูกรีวิวนี้เข้ากับหนังสือที่มี id นี้ ผลลัพธ์เหมือนกับการส่ง `bookId` ตรงๆ

อธิบายขั้นที่ 4 (ดึงข้อมูลพร้อมรีวิว)

`include: { reviews: true }` คือหัวใจของขั้นนี้ ทำให้ Prisma ดึงรีวิวทั้งหมดของหนังสือแต่ละเล่มมาซ้อนอยู่ในฟิลด์ `reviews` (nested list) ตามที่โจทย์ต้องการ

ตัวอย่างผลลัพธ์

```
{
  "id": 1,
  "title": "Clean Code",
  "author": "Robert C. Martin",
  "publishedAt": "2026-11-03T00:00:00.000Z",
  "coverUrl": null,
  "reviews": [
    { "id": 2, "rating": 4, "content": "เนื้อหาแน่น แต่บางบทยาว", "bookId": 1 },
    { "id": 1, "rating": 5, "content": "ดีมาก อ่านแล้วเขียนโค้ดดีขึ้น", "bookId": 1 }
  ]
}
```

ตัวเลือกเสริมที่มีประโยชน์

ต้องการ	เขียนอย่างไร
เรียงรีวิวใหม่สุดก่อน	`reviews: { orderBy: { id: 'desc' } }`
เลือกเฉพาะบางฟิลด์	`reviews: { select: { rating: true, content: true } }`
นับจำนวนรีวิว	`include: { _count: { select: { reviews: true } } }`

รันและทดสอบ

```
node index.js
```

เปิดอีกหน้าต่างเทอร์มินัลแล้วทดสอบ

```
curl -X POST http://localhost:3000/books/1/reviews -H "Content-Type:
application/json" -d '{"rating":5,"content":"หนังสือดีมาก"}'

curl http://localhost:3000/books/1
```

6. ฉบับ Next.js (App Router)

ขั้นที่ 1-2 (แก้ schema และ `db push`) ทำเหมือนหัวข้อ 4 ทุกอย่าง ส่วนที่ต่างคือวิธีสร้างโปรเจกต์ และการเขียน API ในขั้นที่ 3-4

6.1 สร้างโปรเจกต์และติดตั้ง Prisma

```
npx create-next-app@latest book-next
cd book-next
npm install @prisma/client@6
npm install -D prisma@6
npx prisma init --datasource-provider sqlite
```

ตอบคำถามของ `create-next-app` ให้เลือก TypeScript = Yes, App Router = Yes และ import alias เป็น `@/*` (ค่าเริ่มต้น)

จากนั้นทำตามหัวข้อ 4 ต่อได้เลย คือแก้ `prisma/schema.prisma` (เพิ่ม Book + Review), รัน `npx prisma db push` และรัน `node prisma/seed.js` เพื่อใส่ข้อมูลตัวอย่าง

6.2 สร้าง Prisma Client แบบ singleton

ทำไมต้องทำ: ตอนรัน `next dev` โค้ดถูก hot reload บ่อย ถ้าสร้าง `new PrismaClient()` ทุกครั้งจะเปิด connection ซ้ำจนขึ้นเตือน *Too many Prisma Clients* วิธีแก้คือเก็บ instance ไว้ใน `globalThis` แล้วใช้ตัวเดิมซ้ำ

สร้างไฟล์ `lib/prisma.ts`

```
import { PrismaClient } from '@prisma/client';

const globalForPrisma = globalThis as unknown as { prisma?: PrismaClient };

export const prisma = globalForPrisma.prisma ?? new PrismaClient();

if (process.env.NODE_ENV !== 'production') {
  globalForPrisma.prisma = prisma;
}
```

6.3 โครงสร้างไฟล์

Next.js กำหนด URL จาก **โครงสร้างโฟลเดอร์** ไม่ได้ใช้ `app.post('/path')` แบบ Express

```
book-next/
├─ prisma/
│  ├─ schema.prisma
│  └─ seed.js
├─ lib/
│  └─ prisma.ts
└─ app/
   ├─ api/
   │  └─ books/
   │     ├─ route.ts                → GET /api/books
   │     └─ [id]/
   │        ├─ route.ts             → GET /api/books/:id
   │        └─ reviews/
   │           └─ route.ts          → POST /api/books/:id/reviews
   └─ books/
      └─ [id]/
         ├─ page.tsx                → หน้าเว็บรายละเอียดหนังสือ
         └─ ReviewForm.tsx          → ฟอร์มเพิ่มรีวิว
```

6.4 ขั้นที่ 3: API เพิ่มรีวิว

สร้างไฟล์ `app/api/books/[id]/reviews/route.ts`

```
import { NextResponse } from 'next/server';
import { prisma } from '@/lib/prisma';

export async function POST(
  request: Request,
```

```
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const { id } = await params;
    const bookId = Number(id);

    if (!Number.isInteger(bookId)) {
      return NextResponse.json({ error: 'id ไม่ถูกต้อง' }, { status: 400 });
    }

    const body = await request.json();
    const rating = Number(body.rating);
    const content = body.content;

    if (!Number.isInteger(rating) || rating < 1 || rating > 5) {
      return NextResponse.json(
        { error: 'rating ต้องเป็นจำนวนเต็ม 1-5' },
        { status: 400 }
      );
    }
    if (typeof content !== 'string' || content.trim() === '') {
      return NextResponse.json({ error: 'ต้องมี content' }, { status: 400 });
    }

    const book = await prisma.book.findUnique({ where: { id: bookId } });
    if (!book) {
      return NextResponse.json({ error: 'ไม่พบหนังสือ' }, { status: 404 });
    }

    const review = await prisma.review.create({
      data: { rating, content: content.trim(), bookId },
    });

    return NextResponse.json(review, { status: 201 });
  } catch (err) {
    console.error(err);
    return NextResponse.json(
      { error: 'เกิดข้อผิดพลาดที่เซิร์ฟเวอร์' },
      { status: 500 }
    );
  }
}
```

เปรียบเทียบกับ Express

Express	Next.js
`app.post('/books/:id/reviews', ...)`	ไฟล์ `app/api/books/[id]/reviews/route.ts` + ฟังก์ชัน export ชื่อ `POST` (ตัวพิมพ์ใหญ่)
`req.params.id`	อาร์กิวเมนต์ตัวที่ 2 และต้อง `await params` เพราะเป็น Promise
`req.body` (ต้องมี `express.json()`)	`await request.json()`
`res.status(201).json(data)`	`NextResponse.json(data, { status: 201 })`

ตรรกะการตรวจข้อมูล (validate), การเช็กว่ามีหนังสือจริง และ `prisma.review.create` เหมือน Express ทุกอย่าง เพราะส่วนนี้เป็นเรื่องของ Prisma ไม่เกี่ยวกับ framework

6.5 ขั้นที่ 4: API ดึงหนังสือพร้อมรีวิว

ไฟล์ `app/api/books/route.ts` (ดึงทั้งหมด)

```
import { NextResponse } from 'next/server';
import { prisma } from '@/lib/prisma';

export const dynamic = 'force-dynamic';

export async function GET() {
  const books = await prisma.book.findMany({
    include: { reviews: true },
    orderBy: { publishedAt: 'desc' },
  });
  return NextResponse.json(books);
}
```

ไฟล์ `app/api/books/[id]/route.ts` (ดึงเล่มเดียว)

```
import { NextResponse } from 'next/server';
import { prisma } from '@/lib/prisma';

export const dynamic = 'force-dynamic';

export async function GET(
  _request: Request,
```

```
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;
  const book = await prisma.book.findUnique({
    where: { id: Number(id) },
    include: { reviews: { orderBy: { id: 'desc' } } },
  });

  if (!book) {
    return NextResponse.json({ error: 'ไม่พบหนังสือ' }, { status: 404 });
  }
  return NextResponse.json(book);
}
```

อธิบาย

- `include: { reviews: true }` คือหัวใจของขั้นที่ 4 ทำให้ได้ nested list ของรีวิวตามโจทย์
- `export const dynamic = 'force-dynamic'` บอก Next.js ว่าอย่า cache ผลลัพธ์ ไม่เช่นนั้นข้อมูลอาจไม่อัปเดตหลังเพิ่มรีวิวใหม่
- `_request` ขึ้นต้นด้วย underscore เพราะไม่ได้ใช้ตัวแปรนี้ เป็นธรรมเนียมเพื่อไม่ให้ linter เตือน

6.6 ส่วนเสริม: หน้าเว็บแสดงหนังสือและฟอร์มรีวิว

จุดเด่นของ Next.js คือ **Server Component** เรียก Prisma ได้ตรงๆ ไม่ต้องยิง `fetch` ไปหา API ของตัวเอง

ไฟล์ `app/books/[id]/page.tsx`

```
import { notFound } from 'next/navigation';
import { prisma } from '@/lib/prisma';
import ReviewForm from './ReviewForm';

export const dynamic = 'force-dynamic';

export default async function BookPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params;

  const book = await prisma.book.findUnique({
```

```
    where: { id: Number(id) },
    include: { reviews: { orderBy: { id: 'desc' } } },
  });

  if (!book) notFound();

  const avg =
    book.reviews.length > 0
      ? book.reviews.reduce((sum, r) => sum + r.rating, 0) /
book.reviews.length
      : null;

  return (
    <main style={{ maxWidth: 640, margin: '2rem auto' }}>
      <h1>{book.title}</h1>
      <p>โดย {book.author}</p>
      <p>
        คะแนนเฉลี่ย: {avg ? avg.toFixed(1) : 'ยังไม่มีรีวิว'} ({book.reviews.length}
รีวิว)
      </p>

      <h2>รีวิว</h2>
      <ul>
        {book.reviews.map((r) => (
          <li key={r.id}>
            {'★'.repeat(r.rating)}{'☆'.repeat(5 - r.rating)} {r.content}
          </li>
        ))}
      </ul>

      <ReviewForm bookId={book.id} />
    </main>
  );
}
```

ไฟล์ `app/books/[id]/ReviewForm.tsx` (Client Component เพราะมี state และ event)

```
'use client';

import { useState } from 'react';
import { useRouter } from 'next/navigation';

export default function ReviewForm({ bookId }: { bookId: number }) {
```

```
  const router = useRouter();
  const [rating, setRating] = useState(5);
  const [content, setContent] = useState('');
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setLoading(true);
    setError('');

    const res = await fetch(`/api/books/${bookId}/reviews`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ rating, content }),
    });

    setLoading(false);

    if (!res.ok) {
      const data = await res.json();
      setError(data.error ?? 'เกิดข้อผิดพลาด');
      return;
    }

    setContent('');
    setRating(5);
    router.refresh();
  }

  return (
    <form onSubmit={handleSubmit}>
      <h3>เขียนรีวิว</h3>
      <select value={rating} onChange={(e) =>
setRating(Number(e.target.value))}>
        {[5, 4, 3, 2, 1].map((n) => (
          <option key={n} value={n}>{n} ดาว</option>
        ))}
      </select>
      <br />
      <textarea
        value={content}
        onChange={(e) => setContent(e.target.value)}
```

```
        placeholder="เขียนความเห็น..."
        required
      />
      <br />
      <button disabled={loading}>{loading ? 'กำลังส่ง...' : 'ส่งรีวิว'}</button>
      {error && <p style={{ color: 'red' }}>{error}</p>}
    </form>
  );
}
```

การไหลของข้อมูล

1. `page.tsx` (ฝั่งเซิร์ฟเวอร์) ดึง Book พร้อม reviews จาก DB แล้วสร้างหน้าเว็บ
2. ผู้ใช้กรอกฟอร์ม `ReviewForm` (ฝั่งเบราว์เซอร์) ยิง `POST /api/books/:id/reviews`
3. API ตรวจข้อมูลแล้วบันทึกลง SQLite
4. `router.refresh()` สั่งให้ Server Component โหลดข้อมูลใหม่ รีวิวใหม่จึงขึ้นทันทีโดยไม่ต้องรีเฟรชหน้าเอง

6.7 รันและทดสอบ

```
npm run dev
```

เปิดเบราว์เซอร์ที่ `http://localhost:3000/books/1` เพื่อดูหน้าเว็บ แล้วทดสอบ API ด้วย curl

```
curl -X POST http://localhost:3000/api/books/1/reviews -H "Content-Type:
application/json" -d '{"rating":5,"content":"หนังสือดีมาก"}'

curl http://localhost:3000/api/books/1
```

7. กรณีทดสอบที่ควรลอง

การทดสอบเฉพาะกรณีปกติไม่พอ ควรลองกรณีผิดพลาดด้วยเพื่อดูว่า validation ทำงานจริง

กรณี	ข้อมูลที่ส่ง	ผลที่ควรได้
ปกติ	rating 5, content ครบ	201 พร้อมข้อมูลรีวิว
rating เกินช่วง	rating 6 หรือ 0	400
rating ไม่ใช่จำนวนเต็ม	rating 3.5	400

กรณี	ข้อมูลที่ส่ง	ผลที่ควรได้
ไม่มี content	content ว่างหรือไม่ส่ง	400
หนังสือไม่มีอยู่	POST ไปที่ id 9999	404
id ไม่ใช่ตัวเลข	POST ไปที่ id abc	400
ดึงหนังสือ	GET หนังสือที่มีรีวิว	ได้ `reviews` เป็น array ซ้อนอยู่ข้างใน
ลบหนังสือ	ลบ Book ใน Prisma Studio	รีวิวของเล่มนั้นหายตาม (Cascade)

8. ข้อผิดพลาดที่พบบ่อย

อาการ	สาเหตุ	วิธีแก้
Prisma error ว่า relation ไม่ครบ	ลืมใส่ `reviews Review[]` ใน Book หรือ `@relation` ใน Review	เขียนให้ครบทั้งสองฝั่ง แล้วรัน `npx prisma validate`
`Cannot read properties of undefined (reading 'review')`	ยังไม่ได้ `db push` หรือยังไม่ generate Client	รัน `npx prisma db push` และ `npx prisma generate`
`req.body` เป็น `undefined` (Express)	ลืม `app.use(express.json())`	เพิ่มบรรทัดนี้ก่อนประกาศ route
Prisma รายงานว่า id ต้องเป็น Int	ส่ง `req.params.id` ซึ่งเป็น string ตรงๆ	แปลงด้วย `Number()` หรือ `parseInt()`
`params` ใช้ไม่ได้ (Next.js 15+)	`params` เป็น Promise	ใช้ `const { id } = await params`
ขึ้นเตือน Too many Prisma Clients	สร้าง PrismaClient ซ้ำตอน hot reload	ใช้ `lib/prisma.ts` แบบ singleton
เพิ่มรีวิวแล้วหน้าไม่อัปเดต	Next.js cache ผลลัพธ์	ใส่ `export const dynamic = 'force-dynamic'` และเรียก `router.refresh()`
`useState` แล้ว error ใน Next.js	ลืม `'use client'`	ใส่บรรทัดนี้บนสุดของไฟล์ Client Component

อาการ	สาเหตุ	วิธีแก้
import `@/lib/prisma` ไม่เจอ	path alias ไม่ได้ตั้ง	เช็ก `paths` ใน `tsconfig.json` หรือใช้ relative path
ติดตั้ง Prisma แล้วรูปแบบ generator ไม่ตรง	ได้ Prisma 7 ขึ้นไป	ติดตั้งแบบระบุ `@6` ตามหัวข้อ 3

9. เช็กลิสต์ก่อนส่งงาน

- [ ] `schema.prisma` มี model `Review` และ `reviews Review[]` ใน `Book`
- [ ] รัน `npx prisma db push` สำเร็จ และเห็นตาราง Review ใน Prisma Studio
- [ ] POST เพิ่มรีวิวได้ และได้รหัส 201
- [ ] rating นอกช่วง 1-5 หรือไม่ใช่จำนวนเต็ม ถูกปฏิเสธด้วย 400
- [ ] ส่ง id หนังสือที่ไม่มีอยู่ ได้ 404
- [ ] GET หนังสือแล้วมี `reviews` เป็น array ซ้อนอยู่ข้างใน
- [ ] (Next.js) หน้าเว็บแสดงรีวิวและเพิ่มรีวิวใหม่ได้โดยไม่ต้องรีเฟรชเอง

10. สรุป

ขั้น	สิ่งที่ทำ	ไฟล์หรือคำสั่งหลัก
1	เพิ่ม model Review และ relation 1-to-N	`prisma/schema.prisma`
2	ซิงค์ schema กับ SQLite	`npx prisma db push`
3	API เพิ่มรีวิวให้หนังสือ	Express: `POST /books/:id/reviews` / Next.js: `app/api/books/[id]/reviews/route.ts`
4	ดึงหนังสือพร้อมรีวิวซ้อน	`prisma.book.findMany({ include: { reviews: true } })`

แนวคิดสำคัญที่ควรจำคือ ความสัมพันธ์แบบ 1-to-Many ใน Prisma ประกอบด้วยฝั่ง "หลาย" ที่เป็นอาร์เรย์ (`Review[]`) และฝั่ง "หนึ่ง" ที่มี Foreign Key กับ `@relation` ส่วนการดึงข้อมูลที่เกี่ยวข้องมาพร้อมกันใช้ `include` และกฎทางธุรกิจที่ฐานข้อมูลไม่ตรวจให้ เช่น rating 1-5 ต้องตรวจในโค้ด API เอง